Assignment5_Report

April 17, 2026

Assignment#5: Perceptron

Alejandro Barajas

CSCI 485

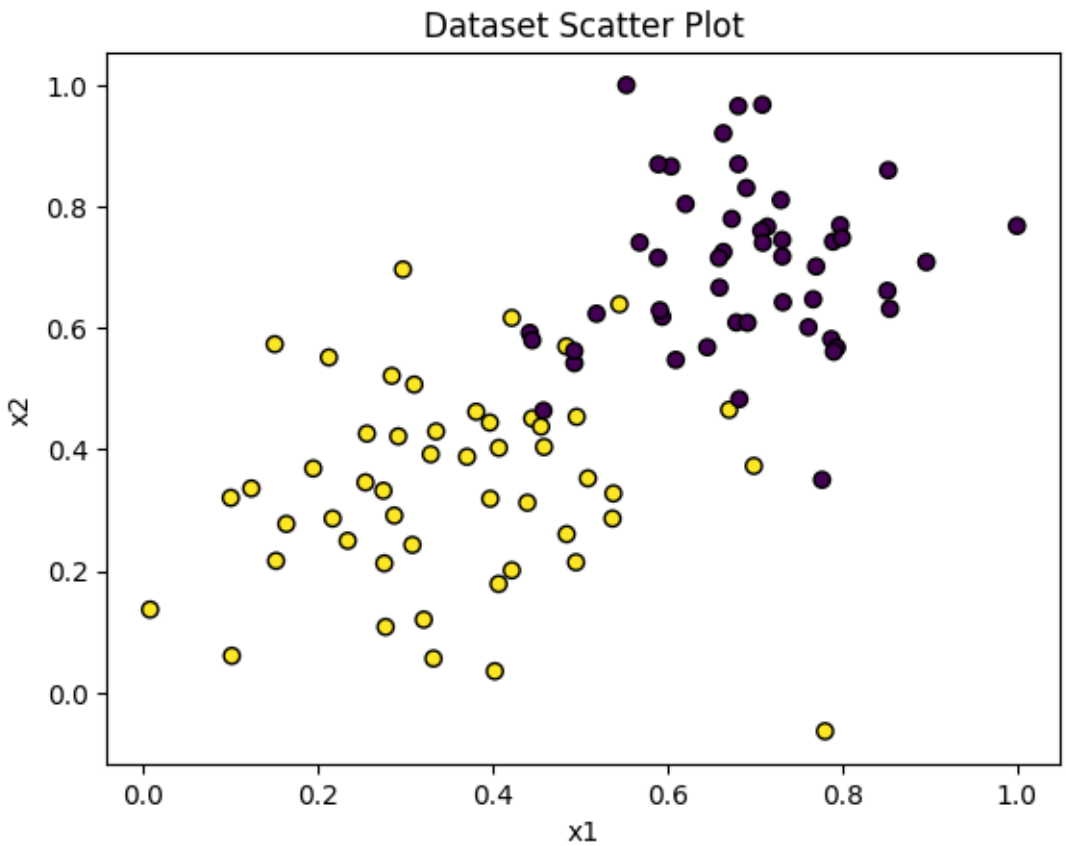
April 17, 2026

1 Introduction

The purpose of this report is to understand perceptrons in a simple heuristic approach and in a gradient descent approach. To do this, I will be using a dataset of three columns with continuous values. Since there are no column names, I had to do a quick cleanup and added three column names: X1, X2, and label. Columns X1 and X2 are continuous values, while the label consists only of 1's or 0's. Below is a table displaying the first 4 elements.

x1	x2	label
0.780510	-0.063669	1
0.287740	0.291390	1
0.407140	0.178780	1
0.292300	0.421700	1
0.509220	0.352560	1

Now that the dataset is cleaned up, I will plot the data to get a better understanding of the distribution of the dataset. The purple data points belong to one classification, while the yellow data points belong to the other. In the graph below, we see that there is an upward trend of data points, which shows the different classifications.



2 Simple Heuristic Approach

This approach consists of three main steps. The first step is to randomly assign weights for each number of data points and a bias term. In my case I will create an array consisting of two random variables because there are only two columns (X1 and X2) and then I will assign a random number as my bias term. The second step is to perform a classification and then a correction of the weights based on the error. The final step is to repeat step 2 enough times to correctly and accurately classify points with minimal error.

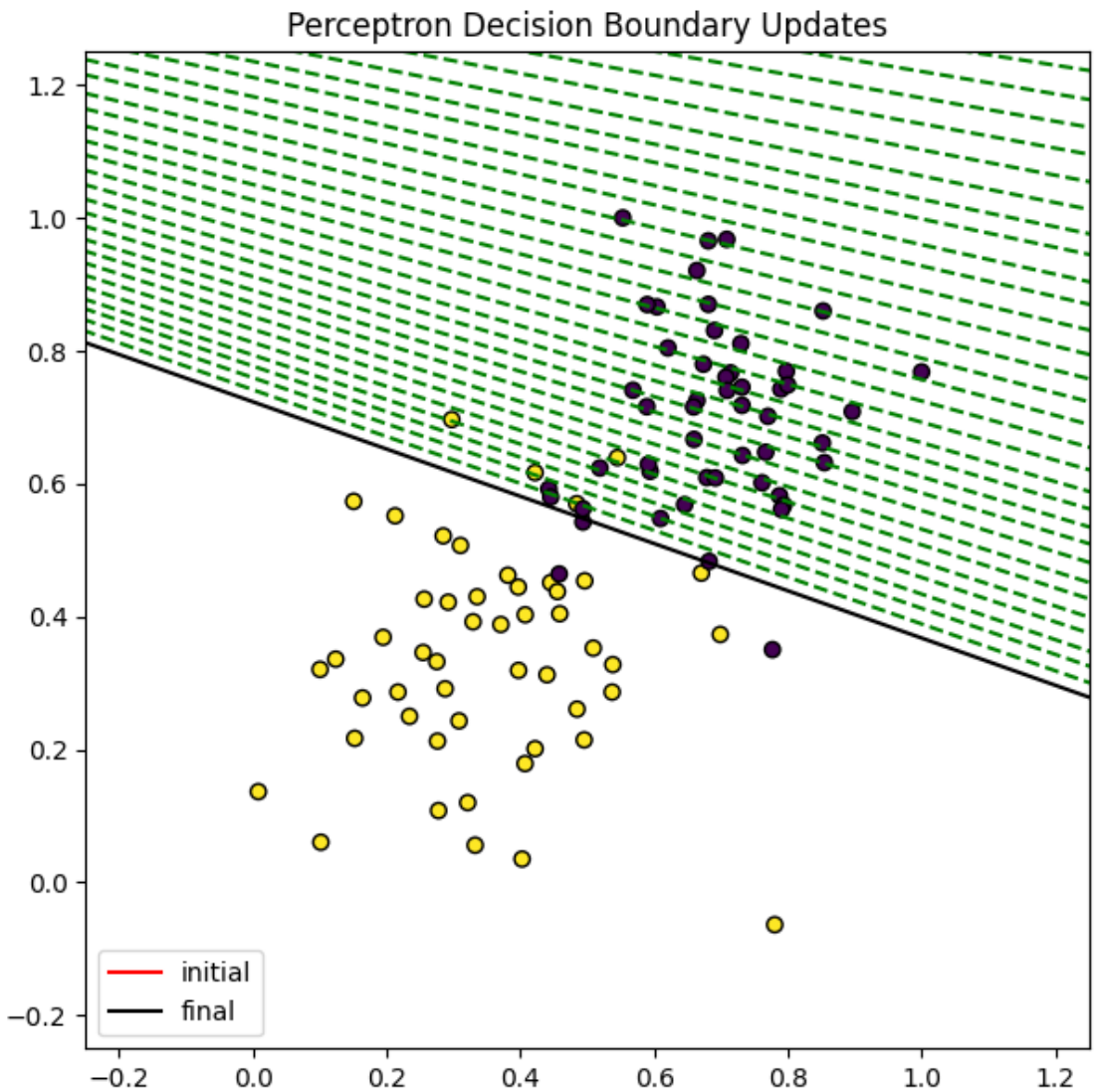
[5]:
```
def perceptron_heuristic(X, y, learning_rate, max_epochs):
    w = np.random.randn(2)
    b = np.random.randn()
    initial_w = w.copy()
    initial_b = b
    lines = []
    error_list = []
    n = len(X)
    for epoch in range(max_epochs):
        for i in range(n):
```

```
            x1 = X['x1'][i]
            x2 = X['x2'][i]
            classification = w[0]*x1 + w[1]*x2 + b
            label_hat = 1 if classification >= 0 else 0
            if (label_hat != y[i]):
                if label_hat == 0:
                    b = b + learning_rate
                    w[0] = w[0] + learning_rate * x1
                    w[1] = w[1] + learning_rate * x2
                else:
                    b = b - learning_rate
                    w[0] = w[0] - learning_rate * x1
                    w[1] = w[1] - learning_rate * x2
                lines.append((w.copy(), b))
                errors = 0

        for i in range(n):
            x1 = X['x1'][i]
            x2 = X['x2'][i]
            classification = w[0]*x1 + w[1]*x2 + b
            label_hat = 1 if classification >= 0 else 0
            if label_hat != y[i]:
                errors += 1

        error_list.append(errors / n)
    return initial_w, initial_b, lines, error_list
```

To deeply understand this approach, it is important to get a visualization of what is happening during the learning process by creating three plots, one for each perceptron of a different leanring rate. To this we can plot an initial line representing the intial seperation line as red. Next we can create a green iteration line representing how each update on the weight impacts the direction in which the line of seperationg moves and by how much. Lastly we will plot a black solid line to represent the final line of seperation, classifiying which data points belong to which label 0 or label 1.

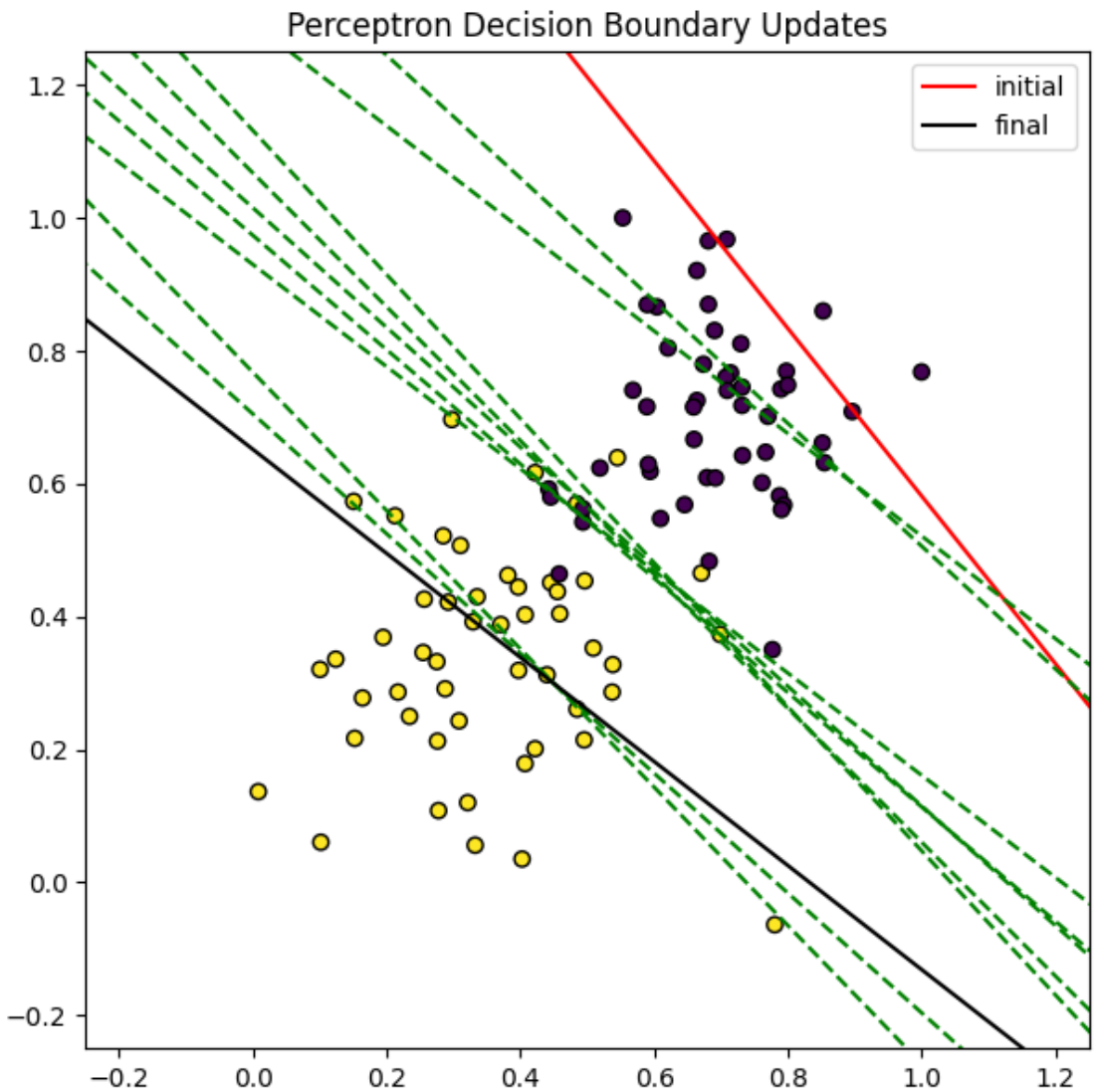
Given that the dataset only has 100 data points, I will chose a epoch value of 3, this is because it will provide a clean plot that will be easy to interpret and still show meaningful movement of the decision boundary.

[13]:
```
initial_w, initial_b, lines, error_list = perceptron_heuristic(X, y,␣
↪learning_rate=0.01, max_epochs=3)
plot_perceptron_training(X, y, initial_w, initial_b, lines)
```



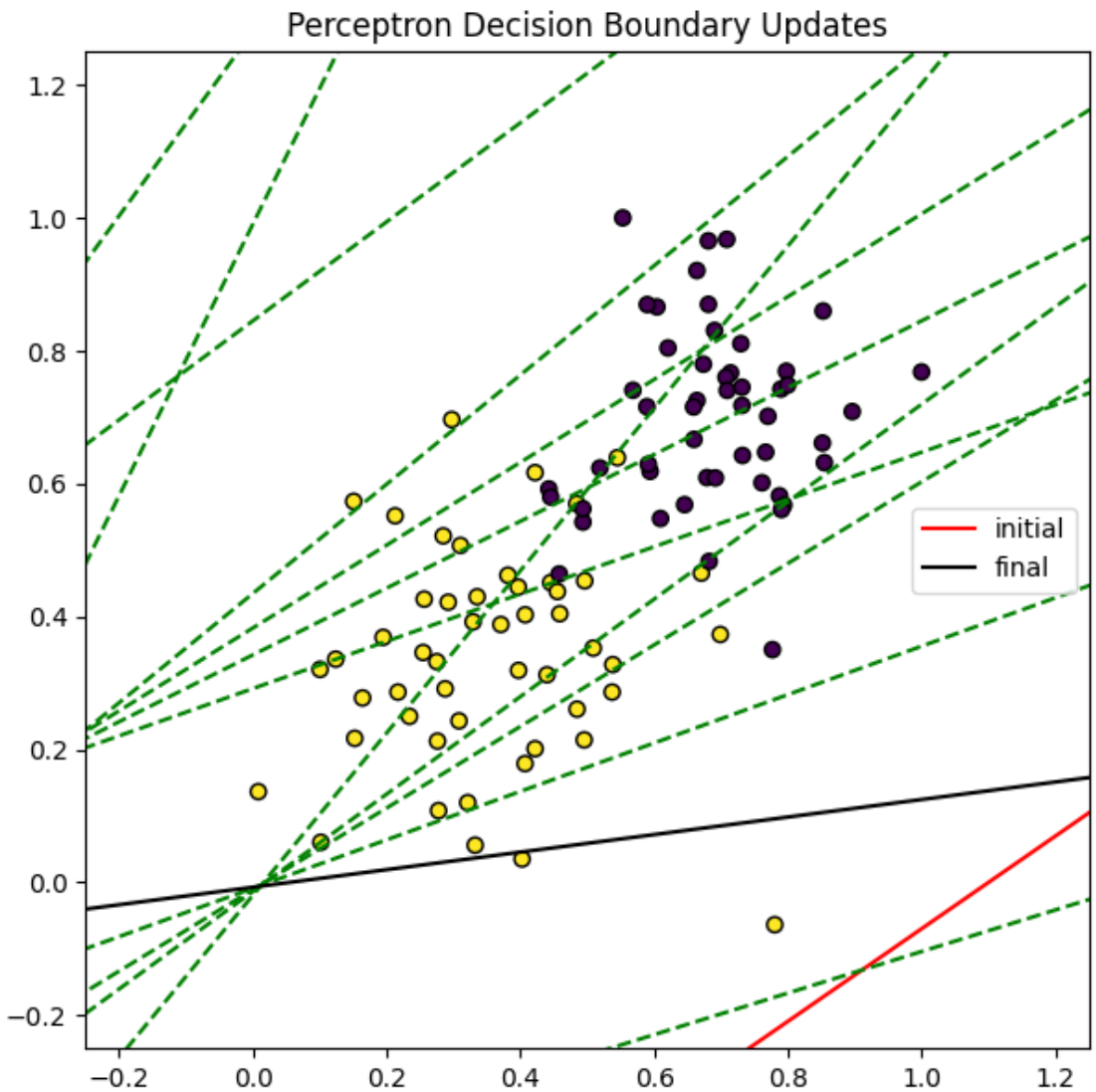
In the graph above, there is some good general seperation between the two different classes in the model with a learning rate of 0.01. We can also see the very small difference between the green lines, this is because of the small learning rate ensuring that we don't over jump the seperation boundary. The downside of a smaller learning rate is that it may need more epochs to show progress.

[14]:
```
initial_w, initial_b, lines, error_list = perceptron_heuristic(X, y,␣
↪learning_rate=0.1, max_epochs=3)
plot_perceptron_training(X, y, initial_w, initial_b, lines)
```



The graph shows poor performance compared to the previous one when the learning rate is 0.1. This is because the training converges faster, resulting in an overshoot of the solution boundary. Comparing it to the previous graph, it is also noticeable that there are fewer green lines.

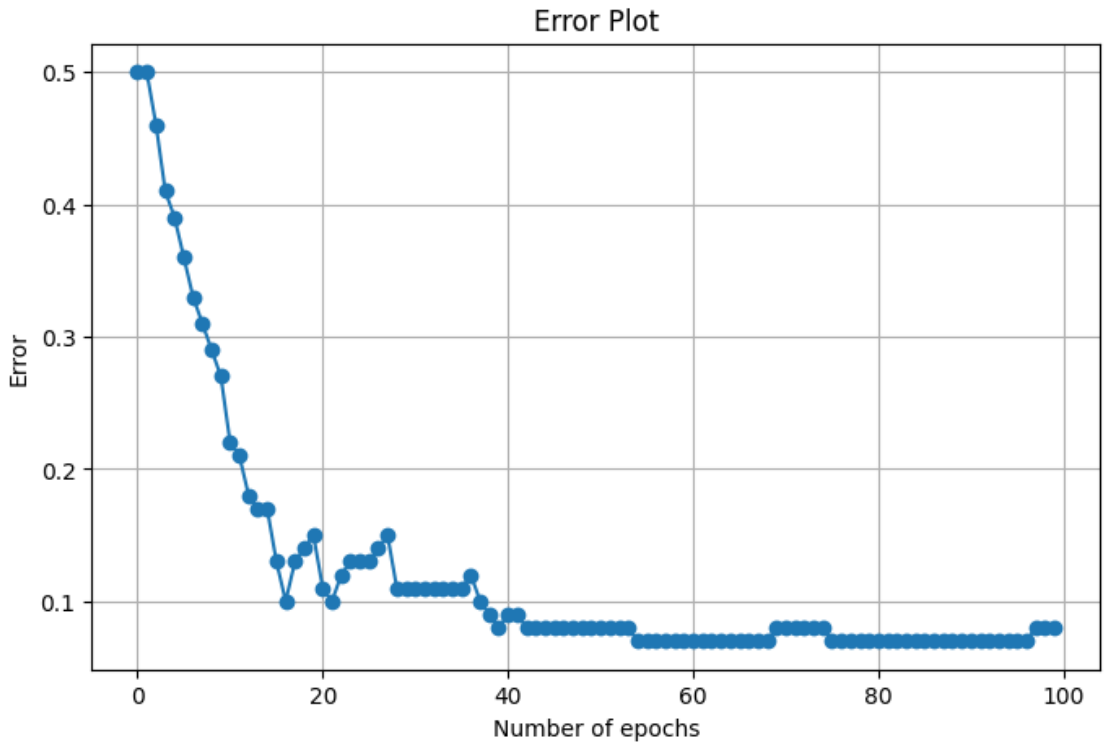
[15]:
```
initial_w, initial_b, lines, error_list = perceptron_heuristic(X, y,␣
↪learning_rate=1.0, max_epochs=3)
plot_perceptron_training(X, y, initial_w, initial_b, lines)
```



This graph shows the worst performance overall when the learning rate is 1.0 as there is an alarming amount of misclassification. This happens because the learning rate is too huge, resulting in a massive overstep of correction for the decision boundary. This method is less stable for 3 epochs

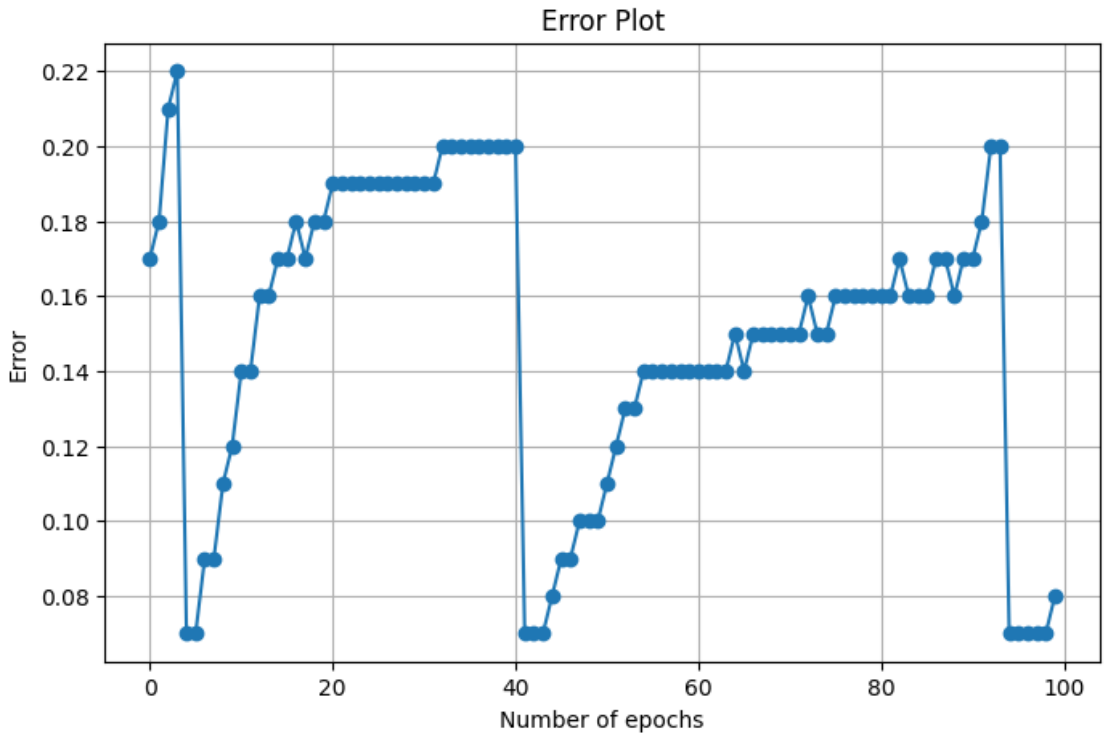
Now that I analyzed a model with different learning rates, it is now important to determine how epochs play a large role for learning rates

[16]:
```
initial_w, initial_b, lines, error_list = perceptron_heuristic(X, y,␣
↪learning_rate=0.01, max_epochs=100)
plot_error_curve(error_list)
```



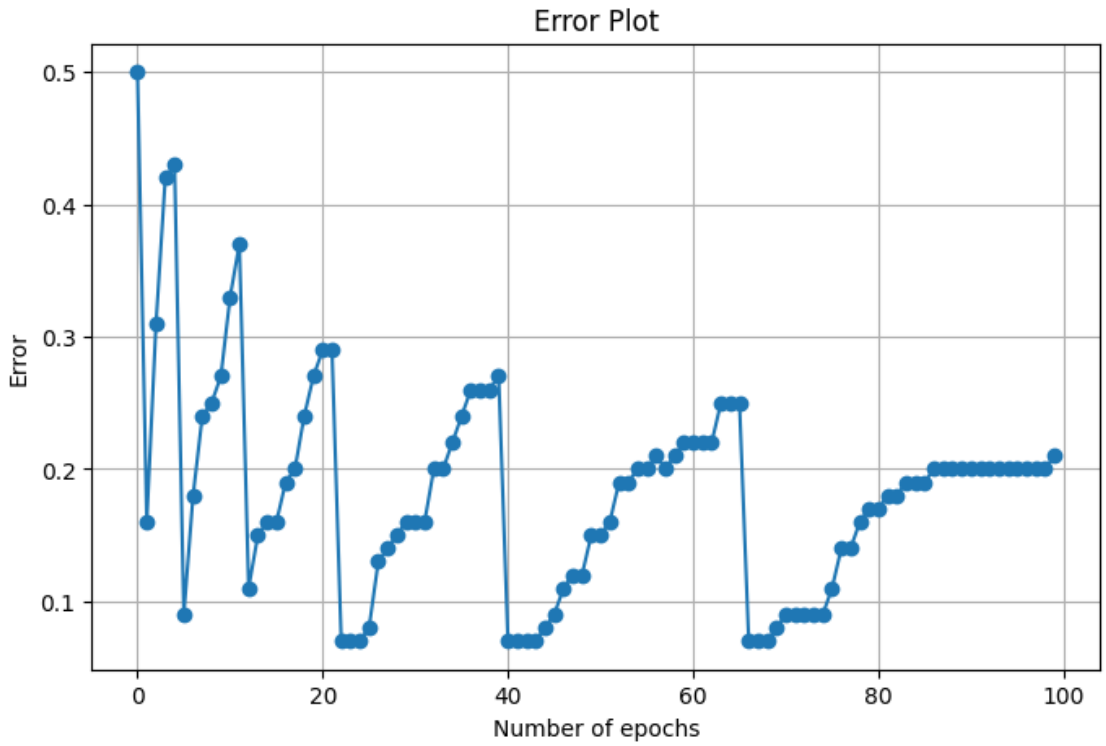
We can see that when the learning rate is 0.01, that it takes a lot of epochs to have minimal error, this happens because a low learning rate converges slowly, but is more stable and controlled compared to a faster/higher training rate.

[25]:
```
initial_w, initial_b, lines, error_list = perceptron_heuristic(X, y,␣
↪learning_rate=0.1, max_epochs=100)
plot_error_curve(error_list)
```



We can see that on the y-axis there is less variance in the error rate compared to the last one. The issue with this learning rate of 0.1, it is clear that there is a periodic jump up in the error rate, signifying how unstable this training method is. We can also notice that the final 100th epoch has a higher error rate compared to the previous graph.

[26]:
```
initial_w, initial_b, lines, error_list = perceptron_heuristic(X, y,␣
↪learning_rate=1.0, max_epochs=100)
plot_error_curve(error_list)
```



In this graph, we can see that there is a similar pattern compared to the previous graph where the learning rate is 0.1, except that in this graph there tends to be more periodic spikes in the error rate. We can also see that on the 100th epoch, this graph produced the highest error rate compared to the others.

3 Conclusion

After some testing and multiple runs, I noticed that for this small dataset of 100 values, a low learning rate is the preferred approach, as it yields stable, controlled convergence to the solution boundary. It is important to note that this could be computationally expensive, since it requires more epochs than a higher learning rate, which requires fewer epochs. One key takeaway from this is the randomness. Since each perceptron starts with random weights and a random b-term, the accuracy of the solution boundary may vary drastically. This analysis does overall capture the general trend and performance for each learning rate.

4 Gradient Descent Approach

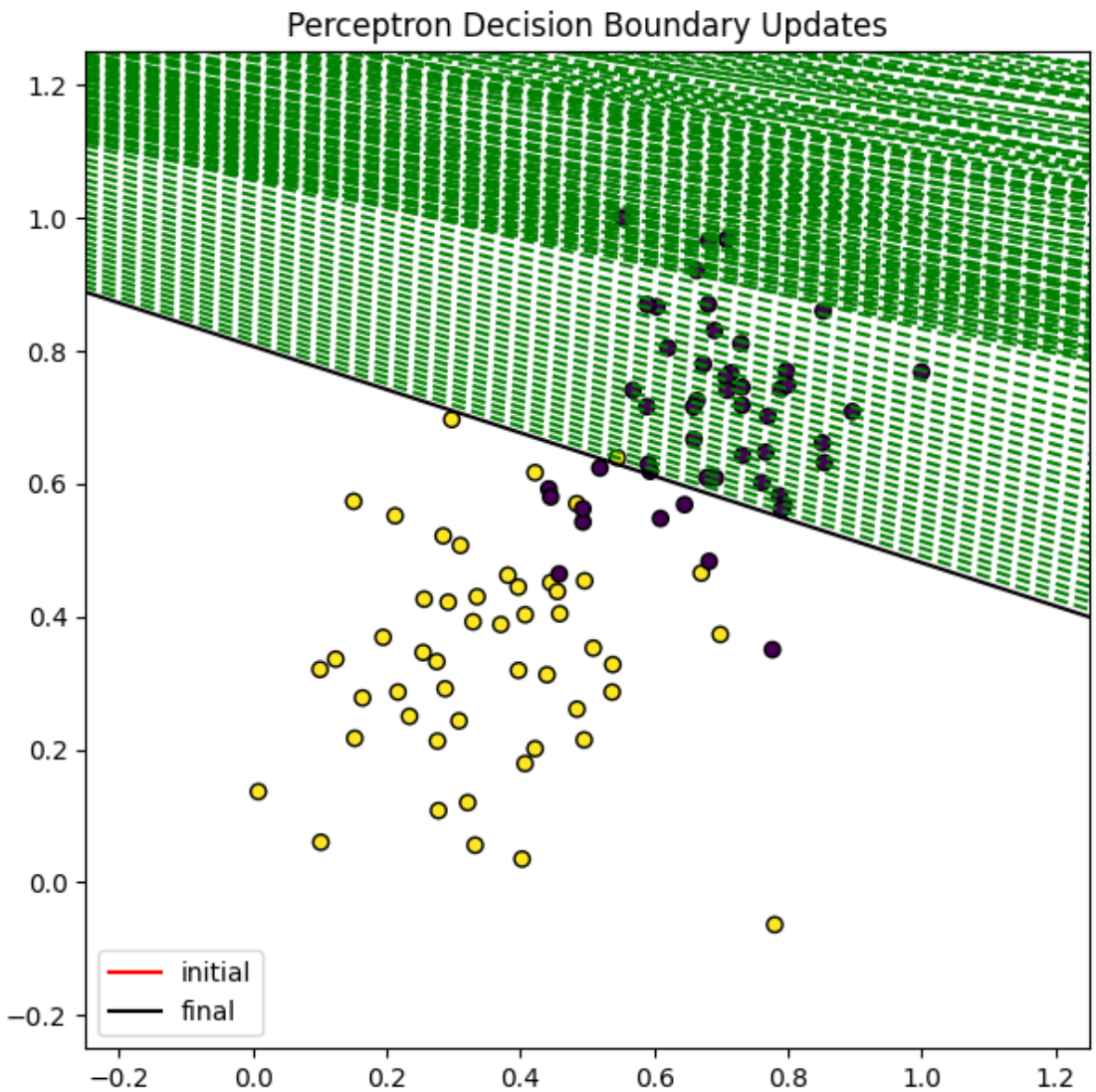
This approach consists of three main steps. The first being to intialize the model with random weights and a random bias term. Tje second being to iterate through every data points and update the model. This conists of computing the prediction with a logistic function, computing the error, updating the bias, and updating each weight. The last step is to repeat the updates until the model stabilizes.

[28]:
```
def perceptron_gradient(X, y, learning_rate, max_epochs=20):
    w = np.random.randn(2)
    b = np.random.randn()
    initial_w = w.copy()
    initial_b = b
    lines = []
    n = len(X)
    error_list = []
    for epoch in range(max_epochs):
        for i in range(n):
            z = w[0]*X['x1'][i] + w[1]*X['x2'][i] + b
            y_hat = 1 / (1 + np.exp(-z))
            error = y[i] - y_hat
            b = b + learning_rate*error
            w[0] = w[0] + learning_rate * error * X['x1'][i]
            w[1] = w[1] + learning_rate * error * X['x2'][i]
            lines.append((w.copy(), b))

        loss = 0
        for i in range(n):
            z = w[0]*X['x1'][i] + w[1]*X['x2'][i] + b
            y_hat = 1 / (1 + np.exp(-z))
            y_hat = np.clip(y_hat, 1e-10, 1 - 1e-10)

            loss += -(y[i] * np.log(y_hat) + (1 - y[i]) * np.log(1 - y_hat))
        loss /= n
        error_list.append(loss)
    return initial_w, initial_b, lines, error_list
```

To deeply understand this approach, it is important to get a visualization of what is happening during the learning process by creating three plots, one for each perceptron of a different leanring rate. To this we can plot an initial line representing the intial seperation line as red. Next we can create a green iteration line representing how each update on the weight impacts the direction in which the line of seperationg moves and by how much. Lastly we will plot a black solid line to represent the final line of seperation, classifiying which data points belong to which label 0 or label 1.

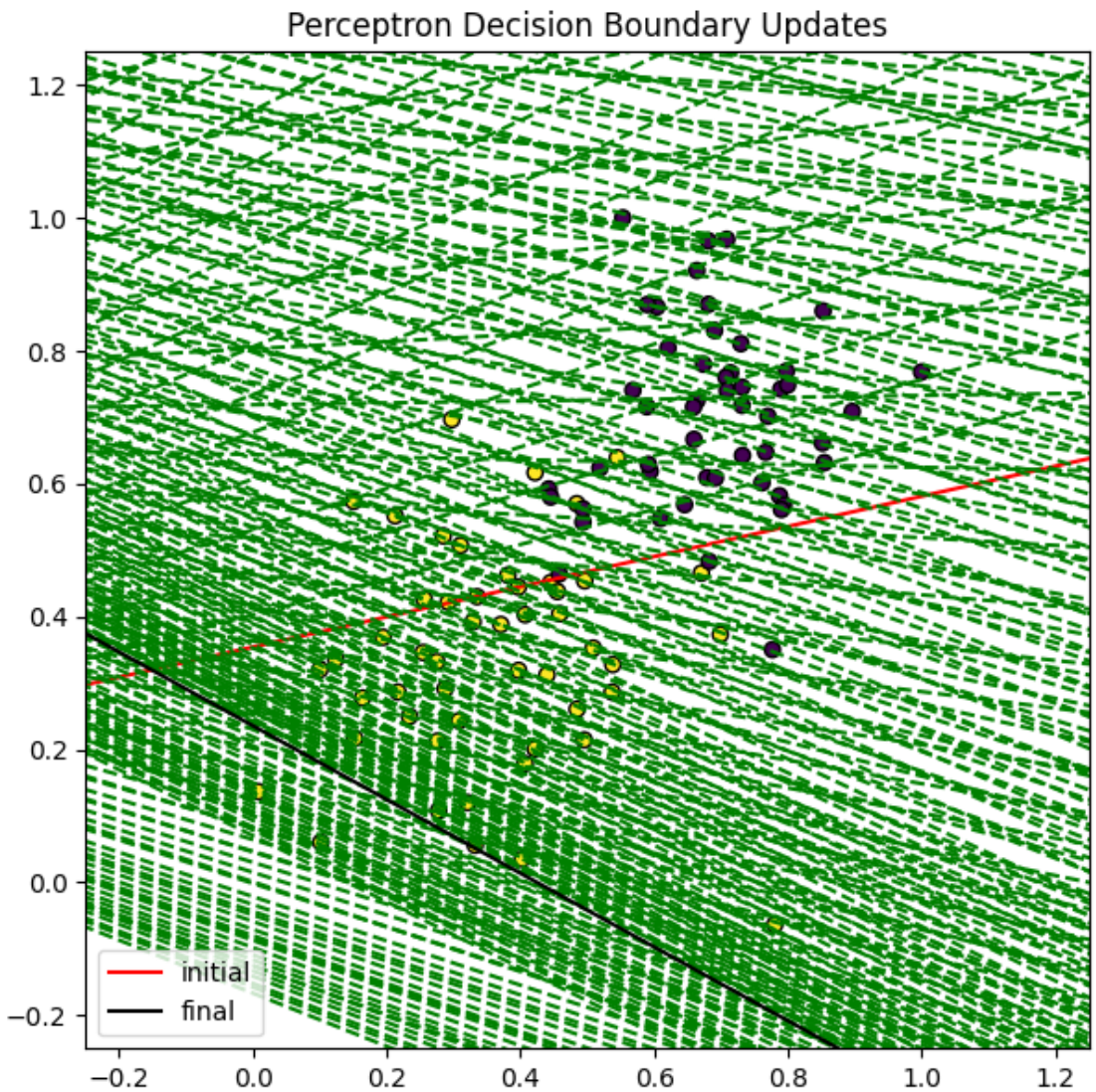
Given that the dataset only has 100 data points, I will chose a epoch value of 3, because we want to do a fair and standard comparison between the two models

[35]:
```
initial_w, initial_b, lines, error_list = perceptron_gradient(X, y,␣
 ↪learning_rate=0.01, max_epochs=3)
plot_perceptron_training(X, y, initial_w, initial_b, lines)
```



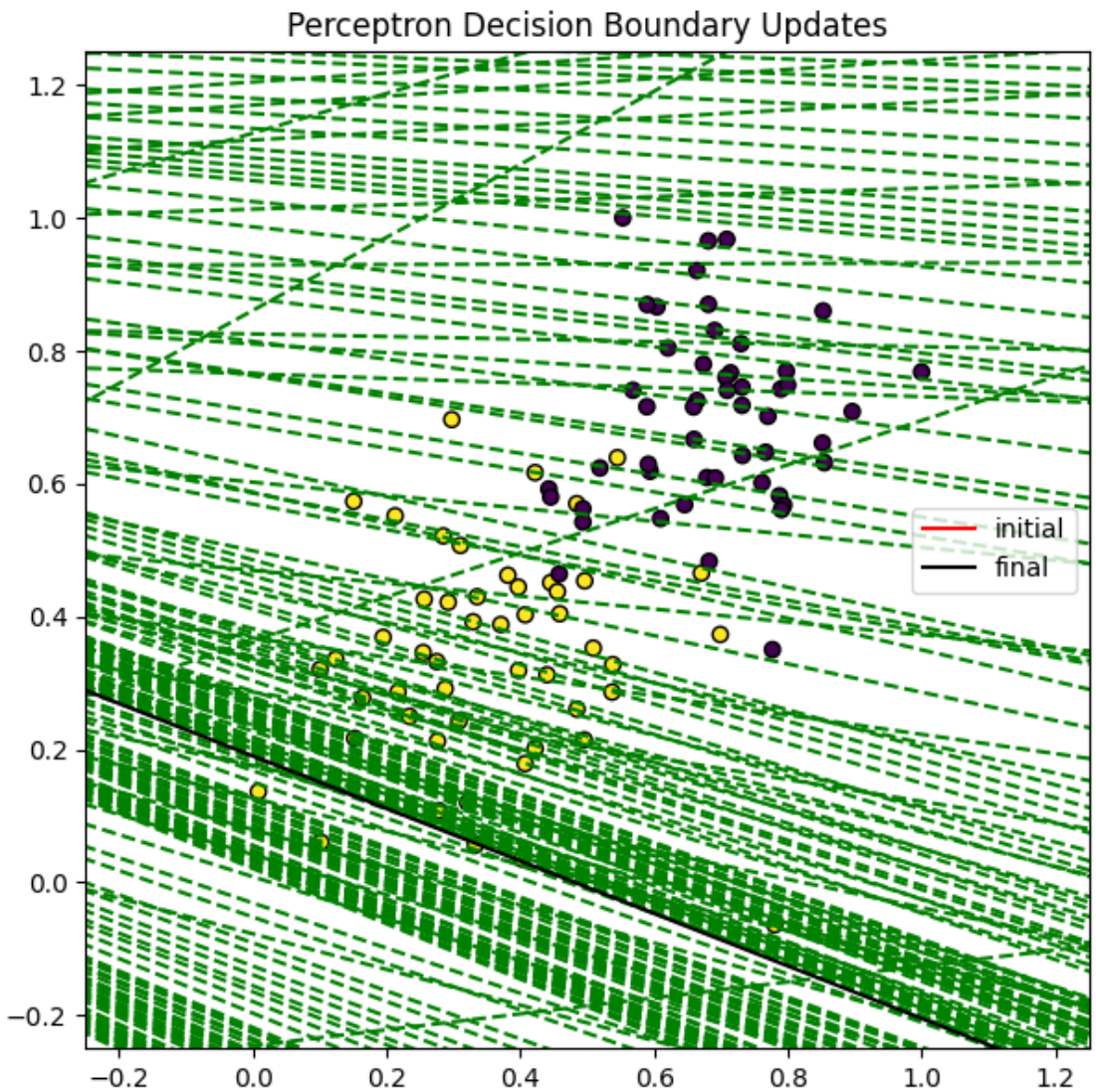
This model does relatively bad job at seperating the data points. The low learning rate can be represented from the low step or change in distance of the green lines.

[36]:
```
initial_w, initial_b, lines, error_list = perceptron_gradient(X, y,␣
↪learning_rate=0.1, max_epochs=3)
plot_perceptron_training(X, y, initial_w, initial_b, lines)
```



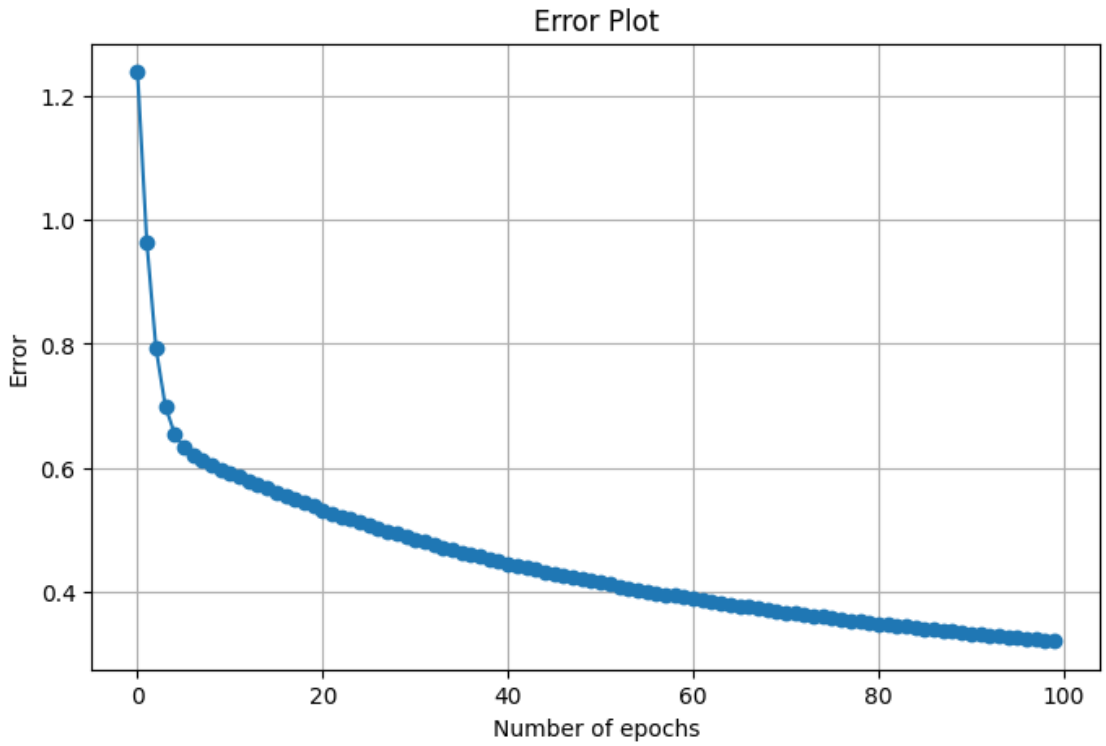
In this gradient descent approach model with a learning rate of 0.1 is a bad approach as it does poorly in the seperation of data points. It is also obvious that there is a lot of noise representing the overshooting or overstepping from the big distances/jumps in the green lines.

[37]:
```
initial_w, initial_b, lines, error_list = perceptron_gradient(X, y,␣
↪learning_rate=1.0, max_epochs=3)
plot_perceptron_training(X, y, initial_w, initial_b, lines)
```



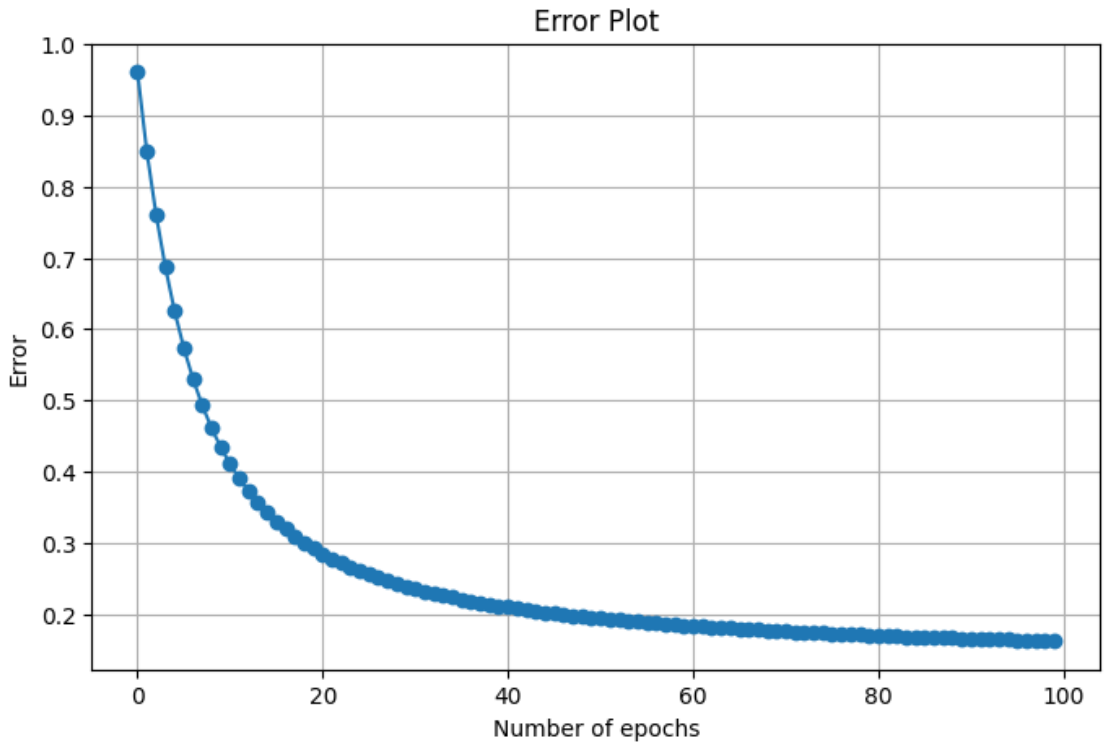
This model with a learning rate of 1.0 resulted in the worst performance. There are is a lot of distances between the lines, signifiying the high learning rate.

[38]:
```
initial_w, initial_b, lines, error_list = perceptron_gradient(X, y,␣
↪learning_rate=0.01, max_epochs=100)
plot_error_curve(error_list)
```



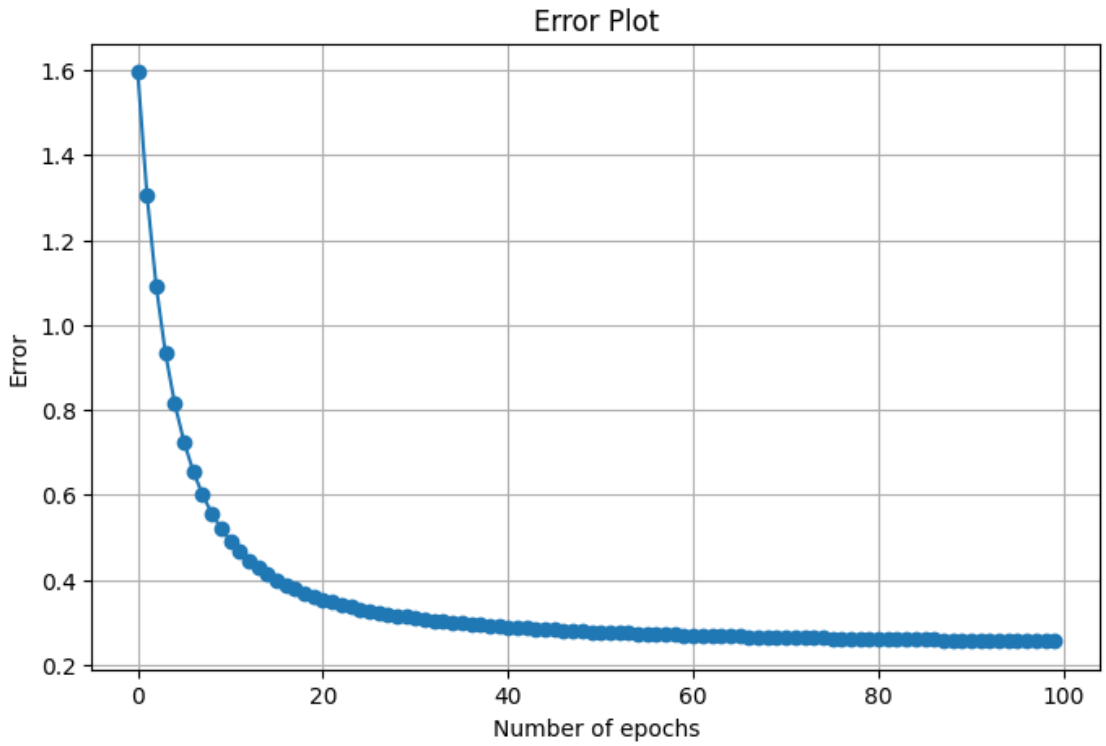
In this error rate plot where learning rate is 0.01, we can see that the model with a low learning rate shows a high error rate initially, but slowly and smoothly decreses to a very low error rate. This comes back to the main benefit of having a low learning rate.

[39]:
```
initial_w, initial_b, lines, error_list = perceptron_gradient(X, y,␣
↪learning_rate=0.1, max_epochs=100)
plot_error_curve(error_list)
```



Suprisingly, we can see the same type of trend from the previous graph in this graph where the model has a learning rate of 0.1. This has different results compared to the simple heuristic perceptron method due to the gradient descent utilization in this version of the perceptron.

[40]:
```
initial_w, initial_b, lines, error_list = perceptron_gradient(X, y,␣
↪learning_rate=1.0, max_epochs=100)
plot_error_curve(error_list)
```



Again suprisingly even with a high learning rate, this version of the perceptron outperforms the simple version the higher the epochs there are, even with a high learning rate.

5 Conclusion

After some testing and multiple runs, I noticed that for this small dataset of 100 values, the learning rate doesn't have much of a signficant impact in the long run compared to the simple preceptron, in this version for a long run of epochs, all learning rates have a controlled descending slope with no periodic spikes. It is important to note that this could be computationally expensive, since it requires more epochs than a higher learning rate, which requires fewer epochs. One key takeaway from this is the randomness. Since each perceptron starts with random weights and a random b-term, the accuracy of the solution boundary may vary drastically intially. This analysis does overall capture the general trend and performance for each learning rate.

6 Comparison

Pros of Heuristic perceptron - Fast updates as there is one pass through on the data, and updates only on mistakes. - Since the perceptron only updates on mistakes, that means it tends to converge in very few epochs. - The boundary adapts quickly

Cons of Heuristic perceptron - Sensitive to data ordering, the order of points affects convergence speed. - The learning rates are really important and sensitive. - Random intialization increases variance so some convergence may take longer or some may take fast. - There is a chance that

there may not even be a seperation due to random initialization

Pros of Gradient Descent perceptron - Since the updates use the logistic function, there is a smooth and stable learning that avoids the jumpiness of the heuristic preceptron. - This model has better generalization, but requires percise epochs. - You update weights per sample giving fast adaptation.

Cons of Gradient Descent perceptron - Slower convergence since you are updating at every sample. - Sensitive to epochs, the higher the epoch the better even if you have a high learning rate. - There is no guarentee of a finding the global minimum, only local.

7 Final Conclusion

Based on my analysis, the simple heuristic perceptron performed well on linearly separable data, but it showed strong sensitivity to the learning rate, especially when the number of epochs was small. Across the 100 epoch error rate plots, the heuristic model displayed a lot of variability because each weight update depends directly on misclassified points and the random initialization of the parameters. On the other hand, the gradient descent model was also sensitive to the learning rate, but its error curves were far more stable regardless of the learning rate; all runs gradually decreased their error by the final epoch. The heuristic perceptron, however, did not consistently converge to a low error, and its performance varied noticeably across different learning rates.

Overall, the analysis suggests that learning rate and epoch count interact differently for the two models. A lower learning rate generally requires more epochs to make meaningful progress, while a higher learning rate can converge faster but risks instability if pushed too far. In a real-world setting, you would first examine the dataset size and whether the underlying pattern is linear or nonlinear, then choose an appropriate learning rate, and finally adjust the number of epochs to ensure sufficient training.

In future work, an improved version of the simple heuristic perceptron could start with zero-initialized weights and a fixed bias term to reduce randomness and allow easier training.